Oliver Krauss

Modern Code - Methods & Modularity II

ModernCode | MTD.ba VZ SS25

Hagenberg 03.11.2025

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH

UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

Why More Methods?

- **Real-world programming**: Most software is built by teams, not individuals
- **Maintainable code**: Good design makes programs easier to fix and improve
- **Problem-solving skills**: Learn to break complex problems into manageable pieces
- **Professional development**: These are industry-standard practices used by software engineers
- **AI collaboration**: Modern developers use AI tools to review and improve their designs
- **Foundation for complexity**: Simple methods become building blocks for complex systems

Review: What We Already Know

- **Method foundations**: Syntax, static methods, signatures, overloading
- **Memory model**: Stack frames, heap objects, pass-by-value
- **Design principles**: Single responsibility, clear naming, proper scope
- **Common mistakes**: Return statements, parameter misuse, unreachable code

Now we learn: How to systematically design complex programs using these building blocks

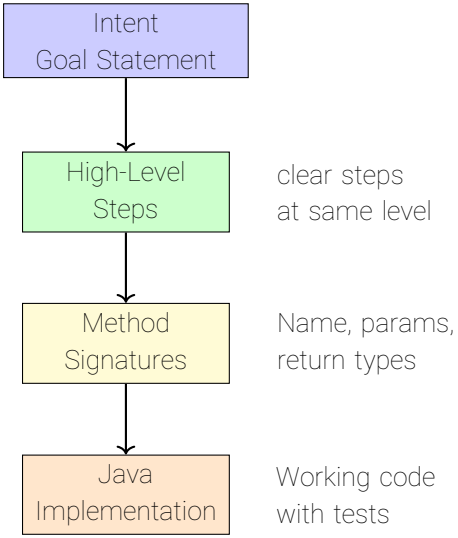
Recap: Method Overloading in Design Patterns

- **Overloading supports good design**: Provide convenience methods with sensible defaults
- **Design principle**: Start simple, add complexity through overloaded variants
- **Example**: `computeDamage(int base)` calls `computeDamage(int base, double crit)` with default crit=1.0
- **Benefits**: Cleaner calling code, backward compatibility, progressive complexity

Stepwise Refinement (Top-Down)

- **Start with intent**: Describe the goal in 1--2 clear lines
- **Identify coherent steps** at one abstraction level
- **Defer details**: Keep each step single-purpose and clear
- **Stop when ready**: When each step becomes mechanical to implement
- **Visual approach**: Think of it as creating a method map

Stepwise Refinement: Visual Flow



What is "Intent"?

- **Intent** = The core purpose of your program
- **Not implementation details**, but the "what" and "why"
- **Should be understandable** to someone who doesn't code
- **Example**: "Run a turn-based combat encounter with multiple actions"
- **Start broad**, then narrow down to specific functionality

From Intent to High Level Steps

```
GOAL: Run a turn-based combat encounter with multiple actions
STEPS:
  INITIALIZE game state (player HP, enemy HP, damage values)
  LOOP through combat turns
    READ player action choice
    PROCESS player action (attack/heal/defend)
    UPDATE game state based on action
    CHECK win/lose conditions
    DISPLAY current status
  END LOOP
  SHOW final result
```

Each step is clear but not yet detailed. We'll refine each one next.

Refine Each Step: Attack & Heal (Pseudocode)

```
REFINE: PROCESS player action (attack/heal/defend)
  IF action == ATTACK THEN
    damage <- computeDamage(baseDamage, critMultiplier,
  enemyArmor)
    newEnemyHp <- max(0, enemyHp - damage)
    displayAttackResult(damage, newEnemyHp)
  ELSE IF action == HEAL THEN
    healAmount <- computeHealing(playerLevel, hasPotion)
    newPlayerHp <- min(MAX_HP, playerHp + healAmount)
    displayHealingResult(healAmount, newPlayerHp)

```

Each substep becomes a method call. We avoid reading global state inside helpers.

Refine Each Step: Defend & Error (Pseudocode)

```
10        ELSE IF action == DEFEND THEN
11          defenseBonus <- computeDefenseBonus(playerLevel, hasShield)
12          displayDefenseMode(defenseBonus)
13        ELSE
14          displayError("Invalid action choice")
15        ENDIF
16
```

Each substep becomes a method call. We avoid reading global state inside helpers.

Refine Update Game State (Pseudocode)

```
REFINE: UPDATE game state based on action
  IF action == ATTACK THEN
    enemyHp <- newEnemyHp
    turnCount <- turnCount + 1
    IF enemyHp <= 0 THEN
      gameState <- "VICTORY"
    ENDIF
  ELSE IF action == HEAL THEN
    playerHp <- newPlayerHp
    turnCount <- turnCount + 1
    potionsUsed <- potionsUsed + 1

```

State updates happen in one place, making the program easier to debug and modify.

Refine Another Step: Game Over Conditions (Pseudocode)

```
1       // Check for game over conditions
2       IF playerHp <= 0 THEN
3         gameState <- "DEFEAT"
4       ELSE IF turnCount >= MAX_TURNS THEN
5         gameState <- "TIMEOUT"
6       ENDIF
7
```

State updates happen in one place, making the program easier to debug and modify.

Single Responsibility Principle

- **What it means**: Each method should have one clear responsibility - one reason to change
- **Why it matters**:
  - Easier to understand and debug
  - Easier to test individual pieces
  - Easier to reuse in different contexts
  - Easier to modify without breaking other parts
- **Example**: Instead of one big `processTurn()` method:
  - `calculateDamage()` - just math
  - `updateHealth()` - just state changes
  - `displayResults()` - just output
- **Test**: If you can describe what a method does with "and" or "or", it probably violates SRP

Decomposition Patterns & Examples I

Calculation helpers (pure functions): return values, no I/O, no side effects

- Scope: Local variables only, no access to external state
- Always same output for same input

Example:

```
1   static int computeDamage(int base, double crit, int armor) {
2       int rawDamage = (int) Math.round(base * crit);
3       return Math.max(1, rawDamage - armor);  // minimum 1 damage
4   }
```

Decomposition Patterns & Examples II

State transformers: take state in, return new/updated state

- Scope: Parameters and locals, no global state modification
- Returns new state without modifying original

Example:

```
1   static int applyHealing(int currentHp, int healAmount, int maxHp) {
2       return Math.min(maxHp, currentHp + healAmount);
3   }
```

Decomposition Patterns & Examples III

I/O and reporting: printing, reading input, formatting output

- Scope: Can access constants but not modify shared state
- Handles user interaction and display

Example:

```
1   static void displayCombatMenu() {
2       IO.println("1. Attack  2. Heal  3. Defend  4. Status");
3   }
```

Decomposition Patterns & Examples IV

Decision makers: analyze state and return choices/decisions

- Scope: Parameters and locals, no state modification
- Returns decisions based on current game state

Example:

```
1   static int determineEnemyAction(int enemyHp, int playerHp) {
2       if (enemyHp < 5) return 2;         // heal if low HP
3       if (playerHp < 10) return 1;       // attack if player weak
4       return (int)(Math.random() * 3) + 1;  // random action
5   }
```

Key principle: Separate concerns to avoid cross-coupling and hidden dependencies

DRY: Don’t Repeat Yourself

- **Principle**: Every piece of knowledge should have a single, unambiguous representation
- **Problem**: Copy-pasted code creates maintenance nightmares
  - Bug fixes must be applied in multiple places
  - Changes require updating every copy
  - Inconsistencies creep in over time
- **Solution**: Extract common logic into reusable methods
- **Example**: Instead of repeating damage calculation everywhere:
  - `computeDamage()` - used by attack, special moves, traps
  - `applyHealing()` - used by potions, rest, level-up
  - `validateInput()` - used by all user input methods

DRY: Before - Repeated Logic

```
1   static void processAttack() {
2     int damage = (int) Math.round(baseDamage * critMultiplier);
3     damage = Math.max(1, damage - enemyArmor);
4     enemyHp = Math.max(0, enemyHp - damage);
5     IO.println("Dealt " + damage + " damage!");
6   }
7   static void processSpecialMove() {
8     int damage = (int)Math.round(baseDamage * 2.0 * critMultiplier);
9     damage = Math.max(1, damage - enemyArmor);
10    enemyHp = Math.max(0, enemyHp - damage);
11    IO.println("Special move dealt " + damage + " damage!");
12  }
```

DRY: After - Single Source of Truth

```
1   static int computeDamage(int base, double multiplier, int armor) {
2    int damage = (int) Math.round(base * multiplier);
3    return Math.max(1, damage - armor);
4   }
5
6   static void processAttack() {
7       int damage = computeDamage(baseDamage, critMultiplier,
        enemyArmor);
8       enemyHp = Math.max(0, enemyHp - damage);
9       IO.println("Dealt " + damage + " damage!");
10  }
11  ...
```

KISS: Keep It Simple, Stupid

- **Principle**: Simplicity should be a key goal in design
- **Problem**: Complex solutions are harder to understand, debug, and maintain
- **Solution**: Prefer straightforward approaches over clever ones
- **When to apply**:
  - Method names should be self-explanatory
  - Logic should be easy to follow step-by-step
  - Avoid unnecessary abstractions for simple tasks
  - Choose readability over performance (unless performance is critical)
- **Example**: Simple boolean logic vs complex nested conditions

KISS: Complex - Hard to Read

```
1  // COMPLEX: Hard to read and understand
2  static boolean canPlayerAct(int playerHp, int playerMp, int
       turnCount, boolean isStunned, boolean isSilenced, int cooldown)
        {
3      return playerHp > 0 &&
4             (playerMp >= 5 || turnCount % 3 == 0) &&
5             !isStunned &&
6             !isSilenced &&
7             turnCount >= cooldown;
8  }
```

This method is hard to read, debug, and maintain due to complex boolean logic.

KISS: Simple Helper Methods - Part 1

```
1   // SIMPLE: Clear, readable, easy to debug
2   static boolean isPlayerAlive(int playerHp) {
3       return playerHp > 0;
4   }
5   static boolean hasMana(int playerMp) {
6       return playerMp >= 5;
7   }
8   static boolean canUseFreeAction(int turnCount) {
9       return turnCount % 3 == 0;
10  }
```

Each helper method has one clear responsibility and is easy to test.

KISS: Simple Helper Methods - Part 2

```
1  static boolean hasResources(int playerMp, int turnCount) {
2      return hasMana(playerMp) || canUseFreeAction(turnCount);
3  }
4  static boolean isOnCooldown(int turnCount, int cooldown) {
5      return turnCount < cooldown;
6  }
7  static boolean hasStatusEffects(boolean isStunned, boolean
       isSilenced) {
8      return isStunned || isSilenced;
9  }
```

Each helper method has one clear responsibility and is easy to test.

KISS: Main Method Using Helpers

```
1  static boolean canPlayerAct(int playerHp, int playerMp, int
       turnCount,
2                              boolean isStunned, boolean isSilenced,
       int cooldown) {
3      return isPlayerAlive(playerHp) &&
4             hasResources(playerMp, turnCount) &&
5             !hasStatusEffects(isStunned, isSilenced) &&
6             !isOnCooldown(turnCount, cooldown);
7  }
```

The main method now reads like English and is easy to understand.

Command-Query Separation

- **Principle**: Methods should either do something OR return something, not both
- **Commands** (do something):
  - Change program state
  - Return `void` or confirmation
  - Examples: `updateHealth()`, `processTurn()`, `displayStatus()`
- **Queries** (return something):
  - Return information without changing state
  - No side effects
  - Examples: `getHealth()`, `isGameOver()`, `computeDamage()`
- **Benefits**:
  - Easier to test and debug
  - Clearer method purpose
  - Safer to call in expressions

Command-Query Separation: Violation

```
1   // VIOLATION: Method does something AND returns something
2   static int attackEnemy(int damage) {
3       enemyHp = Math.max(0, enemyHp - damage);
4       IO.println("Enemy took " + damage + " damage!");
5       return enemyHp;  // Returns new state AND changes it
6   }
```

This method violates the principle by changing state AND returning information.

Command-Query Separation: Solution

```
1  // BETTER: Separate command and query
2  static void attackEnemy(int damage) {
3      enemyHp = Math.max(0, enemyHp - damage);
4      IO.println("Enemy took " + damage + " damage!");
5  }
6
7  static int getEnemyHealth() {
8      return enemyHp;
9  }
```

Now we have clear separation: one method changes state, another returns information.

Naming & Signatures

- **Verb-first names** for actions: `readAction`, `applyDamage`, `renderStatus`
- **Parameter order**: Inputs before configuration, most important first
  - `applyDamage(int targetHp, int damage, int maxHp)` - target and damage are inputs, maxHp is config
  - `computeHealing(int currentHp, int healAmount, int maxHp)` - current state first, then action, then limit
- **Return types** communicate results clearly; avoid boolean "mystery" flags
- **Small parameter lists**: 3-4 parameters maximum; group related values or introduce constants
- **Scope consideration**: Parameters should provide all necessary data without accessing global state

Parameter Grouping: Constants

- **Problem**: Magic numbers scattered throughout code
- **Solution**: Define constants for configuration values
- **Benefits**: Easy to change, self-documenting

```
1  // Use constants for configuration values
2  public static final int MAX_HEALTH = 100;
3  public static final int BASE_DAMAGE = 10;
4  public static final double CRIT_MULTIPLIER = 1.5;
5
6  static int computeDamage(int baseDamage) {
7      return (int) Math.round(baseDamage * CRIT_MULTIPLIER);
8  }
```

Parameter Grouping Strategy 3: Method Overloading

- **Problem**: One method with many parameters
- **Solution**: Overloaded methods with sensible defaults

```
1  // Overloaded methods for different use cases
2  static int computeDamage(int baseDamage) {
3      return computeDamage(baseDamage, CRIT_MULTIPLIER);
4  }
5
6  static int computeDamage(int baseDamage, double critMultiplier) {
7      return (int) Math.round(baseDamage * critMultiplier);
8  }
```

Parameterization

- **Replace magic numbers** with parameters or `public static final` constants
- **Choose defaults at call site**: Avoid hard-coding in helper methods
- **Watch for parameter creep**: Refactor into smaller helpers instead of 6+ parameters
- **Configuration vs Input**: Separate what changes (input) from what's fixed (config)
- **Scope benefits**: Parameters make methods more testable and reusable

Magic Numbers: Before

```
1  // BEFORE: Hard-coded values everywhere
2  static int computeDamage() {
3      return 7;  // Magic number!
4  }
5
6  static void healPlayer() {
7      playerHp = Math.min(100, playerHp + 6);  // Magic numbers!
8  }
```

Hard-coded values make code inflexible and hard to maintain.

Magic Numbers: After

```
1   // AFTER: Configurable and reusable
2   static int computeDamage(int baseDamage, double critMultiplier, int
        armor) {
3       int rawDamage = (int) Math.round(baseDamage * critMultiplier);
4       return Math.max(1, rawDamage - armor);  // Minimum 1 damage
5   }
6
7   static int healPlayer(int currentHp, int healAmount, int maxHp) {
8       return Math.min(maxHp, currentHp + healAmount);
9   }
```

Parameterized methods are flexible and reusable.

Magic Numbers: Constants

```
1  // Usage with constants
2  public static final int BASE_DAMAGE = 7;
3  public static final int HEAL_AMOUNT = 6;
4  public static final int MAX_HEALTH = 100;
5
6  int damage = computeDamage(BASE_DAMAGE, 1.5, 2);  // Call site
       decides values
7  int newHp = healPlayer(playerHp, HEAL_AMOUNT, MAX_HEALTH);
```

Constants provide meaningful names and easy configuration.

Method Map from Pseudocode

```
RUN encounter
  INITIALIZE: playerHp=25, enemyHp=18, baseDamage=7, critMultiplier
    =1.2
  FOR turn from 1 to MAX_TURNS
    action <- readPlayerAction()
    damage <- computeDamage(baseDamage, critMultiplier, enemyArmor)
    enemyHp <- applyAction(action, enemyHp, damage, MAX_HEALTH)
    playerHp <- processEnemyTurn(playerHp, enemyLevel)
    renderCombatStatus(playerHp, enemyHp, turn)
    IF playerHp <= 0 OR enemyHp <= 0 THEN BREAK
  END FOR
  SHOW final result (victory/defeat/timeout)
END
```

From Pseudocode to Method Signatures

- **Goal**: Convert pseudocode steps into Java method signatures
- **Focus**: Method names, parameters, and return types
- **Don't worry about**: Implementation details yet
- **Process**: Design → Signatures → Implementation

Method Signatures: Core Game Logic

```
1  // Core game loop methods - just signatures!
2  static int readPlayerAction()
3  static int computeDamage(int baseDamage, double critMultiplier, int
       armor)
4  static int applyAction(int action, int targetHp, int effectValue,
       int maxHp)
5  static int processEnemyTurn(int playerHp, int enemyLevel)
6  static void renderCombatStatus(int playerHp, int enemyHp, int turn)
```

These signatures show what each method needs and returns, not how it works.

Method Signatures: Game State

```
1  // Game state methods - just signatures!
2  static void initializeGame()
3  static boolean isGameOver(int playerHp, int enemyHp, int turn)
4  static String determineGameResult(int playerHp, int enemyHp, int
       turn)
5  static void displayGameResult(String result, int turn)
```

Simple signatures that focus on the data flow between methods.

Design Benefits of Method Signatures

- **Interface First**: Define how methods interact before implementation
- **Parameter Clarity**: Forces you to think about what data is needed
- **Return Type Planning**: Clarifies what each method produces
- **Testing Strategy**: You can test method contracts before implementation
- **Team Collaboration**: Others can understand your design immediately

Design Checklist

- **Single purpose per method**: Each method does one clear thing
- **Verb names**: Methods start with action words (compute, apply, render)
- **Clear inputs/outputs**: Parameters provide data, return values communicate results
- **Keep shared mutable state out**: Prefer parameters and returns over global variables
- **Choose types thoughtfully**: `int` vs `double`, document units clearly
- **Test helpers in isolation**: Each method should work independently
- **Scope awareness**: Methods should not access variables outside their scope

Testing Your Methods

- **Test each method** in isolation with simple test cases
- **Cover edge cases**: 0, negative numbers, boundary conditions
- **Use assertions** to verify results automatically
- **Test the complete system** with known inputs and expected outputs
- **Start testing early** - easier to fix problems when methods are small
- **Test scope boundaries**: Ensure methods don't access unintended variables
- **Test memory model understanding**: Verify parameter passing behavior

Assertions and Given-When-Then

- **Assertions**: Automatic checks that fail if conditions aren't met
- **Java Assertions**: Disabled by default, enable with `-ea` flag
- **Given-When-Then Pattern**:
  - **Given**: Set up test data and initial conditions
  - **When**: Execute the method being tested
  - **Then**: Verify the results with assertions
- **Benefits**: Tests are self-documenting and fail fast
- **Example**: Given base damage 10, When computing with 1.5x crit, Then result should be 15

Running Java with Assertions

- **Default behavior**: Assertions are ignored (no performance cost)
- **Enable assertions**: `java -ea YourProgram`
- **Enable for specific packages**: `java -ea:com.example... YourProgram`
- **Disable assertions**: `java -da YourProgram`
- **IDE setup**: Configure run configurations to include `-ea` flag
- **Production**: Usually disable assertions for performance
- **Development/Testing**: Enable assertions to catch bugs early

Simple Test Example

```
1   static void testComputeDamage() {
2       // Given: base damage 10, crit multiplier 1.5, no armor
3       int baseDamage = 10;
4       double critMultiplier = 1.5;
5       int armor = 0;
6
7       // When: computing damage
8       int result = computeDamage(baseDamage, critMultiplier, armor);
9
10      // Then: result should be 15
11      assert result == 15 : "Expected 15, got " + result;
12      IO.println("âlJŞ computeDamage test passed");
13  }
```

Complex Interaction Test

```
1   static void testGameTurn() {
2     // Given: game state with player HP 20, enemy HP 15
3     int playerHp = 20;
4     int enemyHp = 15;
5     int baseDamage = 8;
6     int damage = computeDamage(baseDamage, 1.0, 0);
7     // When: player attacks enemy
8     int newEnemyHp = applyAction(1, enemyHp, damage, 100);
9     // Then: enemy HP should decrease by damage amount
10    assert newEnemyHp == (enemyHp - damage) : "Enemy HP wrong";
11    assert newEnemyHp >= 0 : "Enemy HP went negative";
12    IO.println("âlJŞ Game turn test passed");
13  }
```

Edge Case Test

```
1   static void testEdgeCases() {
2       // Given: player at 95 HP, trying to heal 10 points
3       int currentHp = 95;
4       int healAmount = 10;
5       int maxHp = 100;
6
7       // When: applying healing
8       int newHp = applyAction(2, currentHp, healAmount, maxHp);
9
10      // Then: HP should cap at maximum
11      assert newHp == 100 : "Healing should cap at max HP";
12      IO.println("âlJŞ Edge case test passed");
13  }
```

Negative Test

```
1   static void testInvalidInputs() {
2       // Given: invalid action code
3       int invalidAction = 99;
4       int currentHp = 50;
5
6       // When: applying invalid action
7       int result = applyAction(invalidAction, currentHp, 0, 100);
8
9       // Then: should handle gracefully (return unchanged HP)
10      assert result == currentHp : "Invalid action should not change
        HP";
11      IO.println("âŲŞ Negative test passed");
12  }
```

Using AI for Testing

- **AI can help with**:
  - Generating test cases and edge cases
  - Writing test method structures
  - Suggesting assertion strategies
  - Creating test data sets
- **Important Warning**: AI will not understand if your existing code is correct
  - **AI limitation**: It will try to create tests that pass with your current code
  - **Best practice**: Use AI to generate test structure, then manually verify the expected behavior
- **Example**: Ask AI "Generate tests for this method" but verify the assertions yourself

Documentation Basics

- **Use clear, descriptive names** for methods and parameters
- **Add inline comments** for complex logic or non-obvious calculations
- **Document assumptions and constraints**: Parameter ranges, return value meanings, side effects
- **Keep comments up-to-date**: Update when you change the code
- **Good code is self-documenting**: Use comments to explain the "why", not the "what"
- **Document method signatures**: Explain parameter purposes and return value meanings
- **Include examples**: Show how to use the method correctly
- **AI Tip**: AI is excellent at generating Code Documentation comments - use it to help write documentation!

What is JavaDoc?

- **JavaDoc**: Standard documentation format for Java code
- **Purpose**: Generate HTML documentation from code comments
- **Benefits**:
  - Self-documenting code
  - IDE support (hover tooltips, autocomplete)
  - Professional documentation
  - Team collaboration
- **Format**: Special comment block starting with `/**`

JavaDoc Syntax

- **Comment block**: `/** ... */`
- **Description**: First paragraph describes what the method does
- **@param**: Documents each parameter
  - Format: `@param parameterName description`
  - Include constraints, units, valid ranges
- **@return**: Documents the return value
- **@throws**: Documents exceptions (if any)

Documentation Examples

```
1   /**
2    * Calculate damage with critical hit multiplier
3    *
4    * @param baseDamage base damage value (must be non-negative)
5    * @param critMultiplier critical hit multiplier (1.0 = no crit,
        2.0 = double damage)
6    * @return final damage after calculations
7    *
8    * Example: computeDamage(10, 1.5) returns 15
9    */
10  static int computeDamage(int baseDamage, double critMultiplier) {
11      ...
12  }
```

Documentation: Constants and File Headers

```
1   /**
2    * Combat system configuration constants
3    *
4    * These values define the game balance and can be adjusted
5    * to change difficulty and gameplay feel.
6    */
7   public class CombatConfig {
8       /** Maximum health points for any character */
9       public static final int MAX_HEALTH = 100;
10
11      /** Base damage for basic attacks */
12      public static final int BASE_DAMAGE = 10;
13      ...
```

AI Planning: Critique a Method Map

- **Use AI to analyze** your method design before implementing
- **Check for common issues**: naming, parameter order, responsibility separation
- **Get feedback on**: method signatures, data flow, scope usage
- **Learn from AI suggestions**: understand why certain designs are better
- **Iterate and improve**: refine your design based on feedback

AI Planning: Analysis Criteria

- **Method naming**: Are names verb-first and descriptive?
- **Parameter design**: Are parameters ordered logically (inputs before config)?
- **Responsibility separation**: Does each method have one clear purpose?
- **Data flow**: How does data move between methods?
- **Scope usage**: Are methods accessing variables outside their scope?
- **Return values**: Do methods return appropriate results?
- **Constants**: Are magic numbers replaced with named constants?

AI Planning: Prompt Template

Example prompt structure for consistent analysis:

- **Analyze this method map** for a [system description]:
  - Are method names clear and action-oriented?
  - Is parameter order logical (inputs before configuration)?
  - Does each method have a single responsibility?
  - How could the data flow be improved?
  - Are there any scope or state management issues?
  - Suggest specific improvements with code examples
- **Ask for explanations**: Why are certain designs better?
- **Request alternatives**: Show different approaches to the same problem

AI Planning: Expected Feedback

- **Better method names**: More descriptive and action-oriented
- **Improved parameter order**: Inputs first, configuration second
- **Cleaner data flow**: Clear input/output relationships
- **Scope improvements**: Methods don't access unintended variables
- **Constant usage**: Magic numbers replaced with named constants
- **Method decomposition**: Complex methods broken into smaller ones
- **Error handling**: Better parameter validation and edge case handling

Remember: AI provides suggestions, but you must understand and verify them!

Summary: Methods & Modularity II

Design Process

- Stepwise refinement: intent → steps → methods
- Decomposition patterns
- Method mapping: pseudocode to signatures

Design Principles

- Single Responsibility Principle (SRP)
- DRY: Don't Repeat Yourself
- KISS: Keep It Simple, Stupid
- Command-Query Separation

Parameters

- Constants for configuration
- Method overloading
- Magic numbers elimination

Testing

- Given-When-Then pattern
- Java assertions with `-ea` flag
- Test types: simple, complex, edge cases, negative
- AI-assisted testing (with warnings)

Documentation

- JavaDoc syntax and structure
- Parameter documentation
- AI assistance for generation

Key Points

- Systematic approach beats ad-hoc coding
- Good design enables easier testing
- AI tools enhance learning thoughtfully